ADMINISTRATUM-AGENT REFERENCE FORM SPECULUM REVIEW - EN

Status: ADVISORY / SPECULUM REVIEW ONLY

Commit under review: `d7bef1a60b0acbaec9fcde238711764f24fe00a6`

Repository: https://github.com/SoulsLike2313/Imperium-

Generated: 2026-05-18T01:40:10.901437Z

0. Scope and verification

This review targets Administratum-Agent V1 base under `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/` and the supporting target-form / pattern-harvest materials. A local `git clone` was attempted from the sandbox but DNS access to GitHub failed in the container, so runtime execution of the repository was not performed here. HTTP GitHub pages were readable and were used as the evidence source. Therefore this package is a red-team/reference-form design review, not a runtime acceptance receipt.

Evidence basis

• `GitHub commit page` - Commit d7bef1a opens; message is TASK-20260518: implement Administratum Agent V1 base; page reports 71 files changed, +2510/-33.

• `IMPERIUM_NEW_GENERATION/README.md` - New Generation is a CLI-first sandbox; no canon merge, no UI/API integration; first 3 agents have common spine, 7 remain skeletons.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/README.md` - Administratum-Agent mission covers inventory, classification, provenance, dirty runtime detection, useful candidates, merge intelligence, routing. Scope is sandbox-only, no canon fusion, no direct deletion/merge/promotion. Runtime outputs are under RUNS/ADMINISTRATUM_AGENT/<run_id>.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/agent_manifest.json` - Agent is IMPLEMENTED_V1_BASE, stdlib-only, RUNS_LAYER_ONLY, may write own state/memory/reports/receipts/outbox/runs, must not write canon files/other memory/core doctrine, and must not delete, merge, promote, bypass Custodes, ignore dirty state, or claim completion without receipts.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/DESIGN_TARGETS/ADMINISTRATUM_AGENT_V1_TARGET_FORM_20260518/decision_manifest.json` - Design target is TARGET_SPECIFICATION, RU/EN PDFs exist, OSS introduced none, future rich/Textual only considered and not installed/not canon; JSON receipts must remain unaffected.

• `IMPERIUM_NEW_GENERATION/RESEARCH/ARCHITECTURAL_PATTERN_HARVEST/TASK-IMPERIUM-OPEN-SOURCE-AGENT-PATTERN-HARVEST-V0_1/README.md` - Pattern harvest is advisory research only; no external OSS framework is introduced, installed, or canonized. Key outputs include pattern catalog and native contract recommendations.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/TOOLS/administratum_v1_core.py` - Core defines AGENT_ROOT, NEW_GENERATION_ROOT, REPO_ROOT, RUNS_ROOT, SKILL_IDS, UTF-8 JSON writes, classification with zone/artifact/provenance/trust/risk, route_for_object, inventory, dirty-runtime, routing and merge summary.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/TOOLS/administratum_agent_runner.py` - Runner implements status, inventory, classify-path, provenance-index, detect-dirty-runtime, useful-candidates, route-to-organs, merge-summary, check-all; global flags are currently parser-level only, so flag placement after subcommand is not guaranteed.

• `IMPERIUM_NEW_GENERATION/ORGAN_AGENTS/ADMINISTRATUM_AGENT/TESTS/run_check_all.py and runner check-all` - A check-all suite exists and validates manifest, policies, rules JSON, skill manifests, sample path classification, runtime classification, smoke inventory/provenance/routing/merge, CLI

no-color/plain-json, receipts, and no new dirty files outside allowed RUNS.

1. Executive hard verdict

Administratum-Agent V1 is a working first base, not a reference-grade form. It has the right spine: manifest, policies, brain_node, memory directories, skills, CLI runner, core logic, tests, receipts, reports, and ignored RUNS isolation. It must not be cloned into the other nine agents as-is.

Promotion to reference-grade is BLOCKED by missing executable metrics, weak route quality model, non-admitted learning memory, incomplete receipt schema, shallow fake-green semantics, and incomplete CLI UX standardization.

Best next action: harden the reference form before multiplying agents.

2. What is real now

Area	Real V1 base evidence	Red-team status
Manifest	`agent_manifest.json` declares status, permissions, stdlib-only policy, supported skills, forbidden actions.	Good base, not enough schema validation.
Policies	Acceptance, rejection, learning, mutation, provenance, runtime output policies exist.	Too short; not yet executable gates.
Brain node	rules/cases/scoring/vocabulary/doctrine directories exist.	Structure exists; no unit accounting or hygiene gate.
Skills	7 skills listed and skill manifests exist.	Skill contracts need schemas and fixture coverage.
CLI runner	Commands exist for status, inventory, classify, provenance, dirty runtime, useful candidates, route, merge, check-all.	CLI form is useful but not reference-grade.
Runtime isolation	RUNS/ADMINISTRATUM_AGENT is present with gitignore/gitkeep.	Good principle; gate must hard-fail pollution.
Checks	check-all exists and covers smoke paths.	Smoke only; not deep fixture matrix.
OSS discipline	design target and research pack say no OSS installed/adopted; future rich/Textual only advisory.	Correct; keep locked.

3. Hard red-team findings

F-001 - Metrics - BLOCKER_FOR_REFERENCE_GRADE

Finding: No first-class cost/time/resource layer exists. Reports do not yet standardize wall time, CPU time, peak memory, scanned/classified counts, output byte budget, warnings, rejects, route decisions, rerun cost, and human review estimate across every command.

Evidence paths: TOOLS/administratum_agent_runner.py, TOOLS/administratum_v1_core.py, state/current_status.json

Required fix: Add AGENT_METRICS_V0_1 schema and write command_metrics.json, metrics section in every receipt, current_status.metrics, and run_index.jsonl per run.

Gate: metrics_gate must fail if any command lacks metrics fields.

F-002 - KPD usefulness - BLOCKER_FOR_REFERENCE_GRADE

Finding: KPD review exists as a report artifact but no executable scoring model or testable KPD output is part of the agent runtime.

Evidence paths: REPORTS/TASK-20260518-BUILD-ADMINISTRATUM-AGENT-V1-FROM-TARGET-FORM/ADMINISTRATUM_AGENT_V1_KPD_REVIEW_V0_1.md, TOOLS/administratum_agent_runner.py

Required fix: Define kpd_score.json and KPD formula with correctness, actionability, warning value, receipt completeness, false positives, false negatives, runtime cost, dirty-tree behavior, and repeated-work avoidance.

Gate: kpd_gate must require sample fixtures with expected score bands.

F-003 - Routing - BLOCKER_FOR_REFERENCE_GRADE

Finding: route_for_object is too shallow. Default route is MECHANICUS_AGENT when no rule matches; runtime evidence and JSON reports can be routed without Administratum retention, Custodes risk, Inquisition suspicion, or source-context analysis.

Evidence paths: TOOLS/administratum_v1_core.py::route_for_object, brain_node/rules/routing_rules.json

Required fix: Add routing_reason, route_confidence, route_policy_version, primary_owner, co_reviewers, escalation_reason, and fixtures for runtime JSON, receipts, policies, reports, archives, dirty outputs, and private-context references.

Gate: route_quality_gate must compare expected route sets for at least 50 fixtures.

F-004 - Learning safety - BLOCKER_FOR_REFERENCE_GRADE

Finding: Learning memory directories exist, but there is no admission protocol that proves pending learning is reviewed, shadow-tested, admitted, quarantined, pruned, versioned, and reversible.

Evidence paths: memory/correction_memory/owner_corrections.jsonl, memory/learning_queue/pending_learning.jsonl, POLICIES/LEARNING_POLICY.md

Required fix: Introduce LEARNING_ADMISSION_PROTOCOL_V0_1 with PENDING -> CANDIDATE -> SHADOW_VALIDATED -> OWNER_APPROVED -> ADMITTED -> DEPRECATED/QUARANTINED states and admission receipts.

Gate: learning_safety_gate must block any write to permanent memory without admission receipt.

F-005 - CLI UX - WARNING_HIGH

Finding: CLI renderer is useful but not reference-grade. It lacks post-subcommand flag tolerance, compact mode, stable section order contract, severity legend, terminal width handling, and copy-friendly error pages.

Evidence paths: TOOLS/administratum_agent_runner.py::build_parser, DESIGN_TARGETS/.../decision_manifest.json

Required fix: Adopt CLI_VISUAL_UX_STANDARD_V0_1. Flags must work before and after subcommands. Each command must emit header, verdict, warnings, outputs, receipts, metrics, next action, and JSON-equivalent plain mode.

Gate: cli_ux_gate must run command matrix in color/no-color/plain-json/verbose/compact modes on PowerShell-safe examples.

F-006 - Fake green - WARNING_HIGH

Finding: status command can PASS because a snapshot was created, not because the agent is reference-grade. detect-dirty-runtime can return PASS_WITH_WARNINGS; if check-all treats warnings as nonblocking, dirty signals may normalize as acceptable.

Evidence paths: TOOLS/administratum_agent_runner.py::command_status, TOOLS/administratum_v1_core.py::detect_dirty_runtime_outputs

Required fix: Split PASS, WARN, BLOCKED, FAIL semantics. Reference-grade gate must treat specific warnings as blockers depending on command mode and declared acceptance target.

Gate: no_fake_green_gate blocks promotion if any warning has blocker_class=true or unresolved_warning_count>0 for reference-grade claim.

F-007 - Receipt quality - WARNING_HIGH

Finding: Receipts are generated but do not yet carry enough repeatability/provenance detail: command argv, cwd, environment hints, input SHA256, output SHA256, git HEAD, dirty state before/after, metrics, policy versions, and schema version are not consistently mandated.

Evidence paths: TOOLS/administratum_agent_runner.py receipts, receipts/NEW_GENERATION_ADMINISTRATUM_AGENT_V1_IMPLEMENTATION_RECEIPT_V0_1.json

Required fix: Define RECEIPT_SCHEMA_V0_2 and make every command use it. Keep old receipts as historical, not reference-grade.

Gate: receipt_completeness_gate validates every receipt with schema and hashes.

F-008 - Test depth - WARNING_MEDIUM

Finding: check-all is a good V1 smoke gate but too narrow for reference-grade. It lacks golden fixtures, negative fixtures, corruption fixtures, route expectation fixtures, metric expectation fixtures, and repeated-run stability checks.

Evidence paths: TESTS/run_check_all.py, TOOLS/administratum_agent_runner.py::command_check_all

Required fix: Add TEST_FIXTURES with expected outputs, expected warnings, and golden JSON snapshots. Add rerun stability test and output budget test.

Gate: test_depth_gate requires fixture coverage before reference promotion.

F-009 - State and dashboard readiness - WARNING_MEDIUM

Finding: current_status exists but there is no strict dashboard-ready state model with last_checked_at, freshness window, stale status, last_run_id, last_receipt_path, last_blockers, KPD, metric summaries, and actionability.

Evidence paths: state/current_status.json, README.md runtime discipline

Required fix: Define AGENT_CURRENT_STATUS_SCHEMA_V0_2 and dashboard_card.json. Dashboard must read only machine state and receipts, never console text.

Gate: dashboard_readiness_gate fails if stale state can be shown as green.

F-010 - 10k unit growth - WARNING_MEDIUM

Finding: Brain-node directories are present, but there is no unit accounting model. The system cannot tell whether growth is useful knowledge or dirt accumulation.

Evidence paths: brain_node/rules, brain_node/cases, brain_node/scoring, brain_node/vocabulary, memory/*

Required fix: Add AGENT_UNIT_INDEX.json with unit_id, category, source, status, confidence, last_used, usefulness_score, superseded_by, owner_reviewed, and pruning policy.

Gate: memory_hygiene_gate requires unit index and quarantine/prune receipts.

4. Ideal local CLI Organ-Agent form

Reference-Grade Local Organ-Agent Form - V0.1

Commit under review: `d7bef1a60b0acbaec9fcde238711764f24fe00a6`

Status: advisory only. Do not implement from this file without Owner task/gate.

1. Definition

A reference-grade local Organ-Agent is a deterministic, local, script-first, data-first, rules-first brain node. It is not an LLM and not a cloud service. It must inspect, classify, route, measure, reject, report, and learn under controlled admission without mutating canon or other organs.

Reference-grade means reusable form for other Organ-Agents. It does not mean the current Administratum V1 implementation is ideal.

2. Mandatory folder structure

```
ORGAN_AGENTS/<AGENT_ID>/
README.md
agent_manifest.json
role_contract.md
operating_rules.md
POLICIES/
PROTOCOLS/
SCHEMAS/
brain_node/
rules/
cases/
scoring/
vocabulary/
risk_signatures/
doctrine_refs/
memory/
correction_memory/
learning_queue/
rejected_memory/
quarantine/
permanent/
unit_index.json
skills/<skill_id>/
README.md
skill_manifest.json
input_schema.json
output_schema.json
receipt_schema.json
fixtures/
TOOLS/
<agent>_runner.py
<agent>_core.py
TESTS/
```

run_check_all.py
FIXTURES/
GOLDEN/
NEGATIVE/
REPORTS/
templates/
receipts/
state/
current_status.json
dashboard_card.json
outbox/
RUNS/<AGENT_ID>/
.gitignore
.gitkeep

3. Mandatory files

• `agent_manifest.json`: authority, read/write boundaries, supported skills, forbidden operations, dependency policy, status.

• `role_contract.md`: what the agent may decide, what it must escalate, what it cannot own.

• `operating_rules.md`: command semantics, stop behavior, no-fake-green rules.

• `POLICIES/*`: acceptance, rejection, mutation, provenance, runtime output, learning, memory admission.

• `SCHEMAS/*`: report, receipt, route, metrics, KPD, status, learning admission, unit index.

• `brain_node/*`: reviewed units only; no raw generated text as permanent knowledge.

• `memory/*`: correction queue and admitted memory with provenance.

• `skills/*/skill_manifest.json`: stable skill contracts.

• `TESTS/FIXTURES/*`: expected behavior examples.

• `state/current_status.json`: latest truth, stale-aware.

4. Mandatory protocols

1. Truth protocol: exact git head, repo root, command, cwd, dirty state before/after.

2. Runtime protocol: all runtime outputs under ignored `RUNS/<AGENT_ID>/<run_id>/`.

3. Receipt protocol: every command writes schema-valid receipts with hashes and metrics.

4. Route protocol: owner/co-reviewer/escalation, confidence, rationale, rule ids.

5. Learning protocol: pending -> candidate -> shadow validated -> Owner/reviewer approved -> admitted -> deprecated/quarantined.

6. CLI protocol: human-readable terminal output separated from plain JSON.

7. Dashboard protocol: dashboards read machine state and receipts only; stale cannot show green.

8. KPD protocol: usefulness score derived from evidence, not narrative.

5. Required CLI commands

Minimum common commands for every Organ-Agent:

• `status`

• `check-all`

• `inventory` or organ-specific equivalent

• `classify-path` or organ-specific classify

• `route-to-organs` or route check
• `detect-dirty-runtime`
• `metrics-summary`
• `kpd-review`
• `learning-queue`
• `explain-decision`

All commands must support:

• `--plain-json`
• `--no-color`
• `--color`
• `--verbose`
• `--compact`
• flags before and after subcommand where reasonable
• copy-friendly errors

6. Required reports and receipts

Every run must produce:

• `command_report.json`
• `command_metrics.json`
• `receipts/<skill>_receipt.json`
• optional `command_report.md`
• `run_index_entry.json`
• state update candidate, not silent state mutation

7. Reference-grade promotion conditions

Promotion is blocked until:

• every acceptance gate in `REFERENCE_GRADE_ACCEPTANCE_GATES.json` passes;
• route fixture coverage is enough to prevent weak default routing;
• learning admission is controlled;
• KPD is executable;
• CLI modes are stable;
• metrics are emitted by every command;
• no OSS dependency is introduced without a separate Owner-approved receipt.

5. 10k meaningful unit model

A 10k-unit local Organ-Agent is not a neural network. It is a reviewed local knowledge base of deterministic units. The current Administratum-Agent has the first folder shape, but not the accounting, admission, pruning, or usefulness measurement needed for safe growth.

Key rule: more units without admission and pruning equals dirt, not intelligence.

See `AGENT_10K_UNIT_MODEL.json` for the full machine model.

6. Cost/time/resource metrics

Every command must emit:

- wall-clock runtime;
- CPU time best-effort;
- peak memory best-effort;
- files scanned/classified;
- bytes read/written;
- report size;
- receipt count;
- warning/blocker counts;
- rejected input count;
- route decisions;
- useful candidates;
- dirty paths;
- cache hits/misses when cache exists;
- human review estimate;
- rerun cost;
- task cost class.

Write locations: machine JSON, receipt metrics, human summary, current_status, and run index.

7. KPD / usefulness coefficient

KPD V0.1 formula:

`KPD = clamp(0,100, 25*Correctness + 20*Actionability + 15*EvidenceCompleteness + 10*WarningValue + 10*NoFakeGreen + 10*NoDirtyTree + 10*Readability - 15*FalsePositiveRate - 25*FalseNegativeSeverity - 10*CostPenalty - 10*ManualCorrectionPenalty)`

This is intentionally approximate. It is still better than narrative self-assessment. It must be computed from fixtures, receipts, metrics, and manual correction counts.

8. Thinking quality for a non-LLM agent

Thinking quality means behavior quality: classification, routing, rejection, escalation, evidence linking, unknown handling, and stable reruns.

The reference form must measure:

- classification confidence quality;
- route appropriateness;

- risk detection quality;
- evidence linking quality;
- provenance completeness;
- rejection correctness;
- contradiction awareness;
- unknown-handling quality;
- escalation correctness;
- decision explanation quality;
- correction absorption quality;
- stable rerun behavior.

9. CLI visual / UX standard

CLI Visual UX Standard - Local Organ-Agents V0.1

Status: advisory standard for future hardening. CLI only. Do not build AAA UI here.

1. Non-negotiable rules

- Human terminal output and machine JSON output must stay separate.
- `--plain-json` prints only valid JSON, no banners, no ANSI, no human hints outside JSON.
- Color never creates truth. Green appears only when the target-specific evidence is complete.
- WARN is not PASS. PASS_WITH_WARNINGS is not reference-grade green.
- Every command tells the operator: what happened, where output is, what blocked, what to do next.

2. Required modes

Mode	Requirement
default	readable human output with stable sections
--plain-json	machine-only valid JSON
--no-color	no ANSI codes; PowerShell copy-safe
--color	explicit color mode, never forced
--verbose	includes details and expanded evidence
--compact	one-screen summary for routine commands

3. Flag placement tolerance

Reference-grade CLI must accept common safe forms:

- `python runner.py --no-color status`
- `python runner.py status --no-color`
- `python runner.py --plain-json route-to-organs --path X`

• `python runner.py route-to-organs --path X --plain-json`

If a flag is not accepted after subcommand, the error must be explicit and include the corrected command.

4. Stable output sections

Human mode section order:

1. Header
2. Run identity
3. Verdict block
4. What happened
5. Warnings/blockers
6. Outputs and receipts
7. Metrics
8. Next action
9. Optional details

5. Verdict semantics

Verdict	Meaning	Color
PASS	complete for declared target, no blockers	green
WARN	usable output, unresolved nonblocking warnings	yellow
PASS_WITH_WARNINGS	allowed only for V1 helper output, not reference promotion	yellow
BLOCKED	cannot proceed until fix	red
FAIL	command/test failed	red
REJECT_*	refused unsafe or forbidden request	red
UNKNOWN	evidence insufficient	cyan/neutral

6. Error messages

Bad error: `invalid option`.

Reference-grade error:

```
[BLOCKED] Unknown flag placement: --no-color after subcommand.
Accepted forms:
python runner.py --no-color status
python runner.py status --no-color
No files were mutated. No receipt was written because parsing failed.
```

7. Windows PowerShell rules

• Avoid terminal-only glyphs as required truth markers.

• Provide ASCII fallback for tables and borders.

• Do not require ANSI to understand status.

• Paths must be copy-friendly.

• UTF-8 only; no mojibake allowed.

8. Output budgets

• Default output should fit into one or two screens for routine status.

• Large detail blocks require `--verbose` or report path.

• Tables must be aligned but not wide enough to force constant horizontal scrolling.

9. No AAA UI here

CLI polish is allowed. Full visual dashboard/UI is deferred. The terminal standard is about comfort and truth, not animation.

10. What must NOT be copied from current V1

Do not copy	Why	Required replacement
Default route to MECHANICUS when uncertain	Hides ownership ambiguity.	UNKNOWN/NEEDS_REVIEW plus owner/co-reviewer mapping.
Narrative KPD only	Cannot drive gates.	Executable KPD JSON and fixture bands.
Smoke check-all as promotion proof	Too shallow.	Fixture matrix and rerun stability.
PASS for status snapshot creation	Fake-green risk.	Status must report readiness target and unresolved blockers.
PASS_WITH_WARNINGS as acceptable promotion state	Warning blindness.	Target-specific warning severity policy.
Direct memory files without admission state machine	Absorbs dirt.	Learning admission and unit index.
CLI global flags only	Operator friction.	Flag placement tolerance.
Receipts without full reproducibility fields	Audit gaps.	Receipt V0.2 with command/git/hashes/metrics.

11. What to strengthen first

1. Fake-green semantics and warning severity.
2. Receipt V0.2 schema.
3. Metrics layer.
4. Route quality model and fixtures.
5. Learning admission and memory hygiene.
6. KPD/self-review.
7. CLI UX polish.
8. Dashboard readiness.

CLI UX matters, but metrics/receipts/routes must precede visual polish; otherwise pretty terminal output becomes a mask.

12. Roadmap

Administratum-Agent Hardening Roadmap - V1 Base to Reference Grade

Status: advisory only. Do not implement without a registered task.

Stage	Goal	Files affected	Tests needed	Outputs	Risk	Acceptance
Immediate fixes	Remove V1 friction and fake-green vectors	runner, check-all, status	flag placement, warning severity	flag receipt, no-fake-green receipt	parser churn	flags stable; warnings not hidden
V1.1 schemas	Standardize receipts/status/routes/errors	SCHEMAS, core, skills	schema fixtures	schema guard receipt	over-schema	every output validates
V1.2 metrics	Cost/time/resource layer	core, runner, status, run index	metrics presence	command_metrics.json	Windows metric gaps	nulls require reason
V1.3 learning	Controlled correction memory	memory, policies, schemas	admission state machine	learning receipt, unit index	absorbing dirt	no permanent memory without admission
V1.4 KPD	Executable usefulness score	schemas/kpd, tools, fixtures	score bands	kpd_score.json	vanity scoring	reproducible score from evidence
V1.5 CLI UX	Pleasant terminal without truth loss	runner, CLI standard	mode matrix	cli_ux_receipt	polish hiding truth	plain-json clean; human output stable
V1.6 dashboard readiness	Dashboard-readable truth	state, dashboard_card, schemas	stale state fixtures	dashboard_card.json	dashboard source of truth	stale cannot show green
V2 reference gate	Promote reusable form	all gates	full gate suite	promotion candidate receipt	copying weak form	Owner accepts after gates

First hardening order

1. Fake-green and CLI flag placement.
2. Receipt schema V0.2.
3. Metrics model V0.1.
4. Route quality fixtures and routing confidence.
5. Learning admission protocol.
6. KPD/self-review.
7. CLI visual comfort.
8. Dashboard readiness.

Do not copy Administratum-Agent V1 into other agents before at least stages 1-4 are done.

13. Acceptance gate matrix

See `REFERENCE_GRADE_ACCEPTANCE_GATES.json`. Gates are blocking for reference promotion, not necessarily for V1 helper use.

Critical gates:

• truth gate;

- clean tree gate;
- runtime isolation gate;
- CLI UX gate;
- metrics gate;
- KPD gate;
- route quality gate;
- learning safety gate;
- memory hygiene gate;
- receipt completeness gate;
- no OSS dependency gate;
- no canon fusion gate.

14. Next recommended Servitor tasks

Task	Purpose	Files likely touched	Acceptance
TASK-ADMIN-AGENT-V1_1-CLI-FAKEGREEN	Fix status semantics, warning severity, and flag placement.	runner, tests, CLI standard	flags before/after subcommand; status cannot imply reference-grade.
TASK-ADMIN-AGENT-V1_1-RECEIPT-SCHEMA	Define receipt V0.2 and migrate commands.	SCHEMAS, core, runner	every receipt validates and includes git/command/hashes/metrics.
TASK-ADMIN-AGENT-V1_2-METRICS	Add command metrics layer.	core, runner, state	every command emits metrics and cost class.
TASK-ADMIN-AGENT-V1_2-ROUTE-FIXTURES	Strengthen routing rules and fixtures.	brain_node/rules, tests fixtures	route quality gate passes on runtime/report/policy/script/dirty fixtures.
TASK-ADMIN-AGENT-V1_3-LEARNING-ADMISSION	Add correction memory admission protocol.	memory, policies, schemas	no permanent memory without admission receipt.

15. Final verdict

- Current V1 base: ACCEPTED AS WORKING BASE ONLY.
- Reference-grade form: BLOCKED.
- Other 9 agents: DO NOT CLONE THIS FORM YET.
- OSS: NONE introduced; keep advisory only.
- Most important blockers: metrics, route quality, learning admission, receipt completeness, fake-green semantics.